

CSD3009 - DATA STRUCTURES AND ANALYSIS OF ALGORITHMS

DR.S.DEVARAJU

Module 2

Stacks and Queues: Array- 1D and 2D array, Stack - Applications of stack: Expression Evaluation - Conversion of Infix to postfix and prefix expression, Tower of Hanoi. Stack ADT – Array based Stacks, Linked Stacks – Implementing Recursion using Stacks, Queues – ADT, Array based Queue, Linked Queue, Double ended queue, Circular queue.

<https://www.javatpoint.com/data-structure-tutorial>

Array

One-Dimensional Array

- A linearly organized **collection of identical data type elements** is referred to as a **One-Dimensional (1D) array**. A 1D array's items are kept in a single and **continuous region in memory**, and each element is accessible via its **index**. An array's initial element always has an **index 0**, and its last element always has an **index n-1**, where n is the array's size.
- Declaration of One-Dimensional Array
- A 1D array can be declared as follows in C:
- **data_type array_name[array_size];**

One-Dimensional Array

- For illustration, the following sentence can be used to declare an integer array with a size of 5.
- **int arr[5];**
- After an array is declared, we can use the index notation to access its elements:
- `arr[0] = 1;    // Assign 1 to the first element`
- `arr[1] = 2;    // Assign 2 to the second element`

One-Dimensional Array

Advantages

- They are **easy to learn and use**.
- Comparatively speaking, they need **less memory** than 2D arrays.
- They can be accessed more **quickly** because only one index is needed.

Disadvantages

- They cannot **accurately** represent intricate data structures.
- In the case of **larger datasets**, they can be challenging to read and analyze.

Two-Dimensional Array

- A **2D array** is a group of identical data-type components arranged in a grid-like pattern. Essentially, it is an array of arrays, where each array corresponds to a **grid row**. A **2D array's** items are kept in a single and **continuous area in memory**, and each one may be reached using its **row** and **column indices**. The row number is represented by the first index, and the column number is represented by the second index.
- Declaration of Two-Dimensional Arrays
- A 2D array can be declared as follows in C:

```
data_type array_name[row_size][column_size];
```

Two-Dimensional Array

- For instance, the following sentence can be used to declare a 2D integer array with the dimension **3x3**.

```
int arr[3][3];
```

- The **row** and **column** index notation is used to access the elements of a 2D array once it has been declared:

```
arr[0][0] = 1; // At row 0 and column 0
```

```
arr[1][2] = 3; // At row 1 and column 2
```


Two-Dimensional Array

Advantages

- They can represent complex data structures like *matrices* and *tables*.
- They offer an **effective method** for grid-like data storage and manipulation.
- **For larger datasets**, they can **be quickly visualized** and analyzed.

Disadvantages

- When compared to 1D arrays, they require **more memory**.
- They require **two indices**, so accessing takes longer.

1D and 2D arrays have different properties

- main differences between one-dimensional and two-dimensional arrays are as follows:

1. Syntax:

- In C, there are differences in the syntax for declaring and accessing the elements of 1D and 2D arrays. When accessing the elements of a **1D array**, we use a **single index**, whereas when accessing the elements of a 2D array, we use **two indices**.

2. Memory:

- As **1D arrays** store elements linearly as opposed to 2D arrays, they use less memory. However, as **2D arrays** store data in a **grid-like format**, where each row is saved as a separate 1D array, they demand more memory.

1D and 2D arrays have different properties

3. Access time:

- Because a 1D array only requires **one index** to reach an element, it can be accessed more quickly than a 2D array. In comparison, using **2D arrays** requires **two indices**, which could slow down the procedure.

4. Complexity:

- **2D arrays** can be more **challenging**, especially for beginners. In contrast, **1D array** are easier to use and comprehend. Understanding row and column indices is necessary for using 2D arrays which can be difficult to grasp at first.

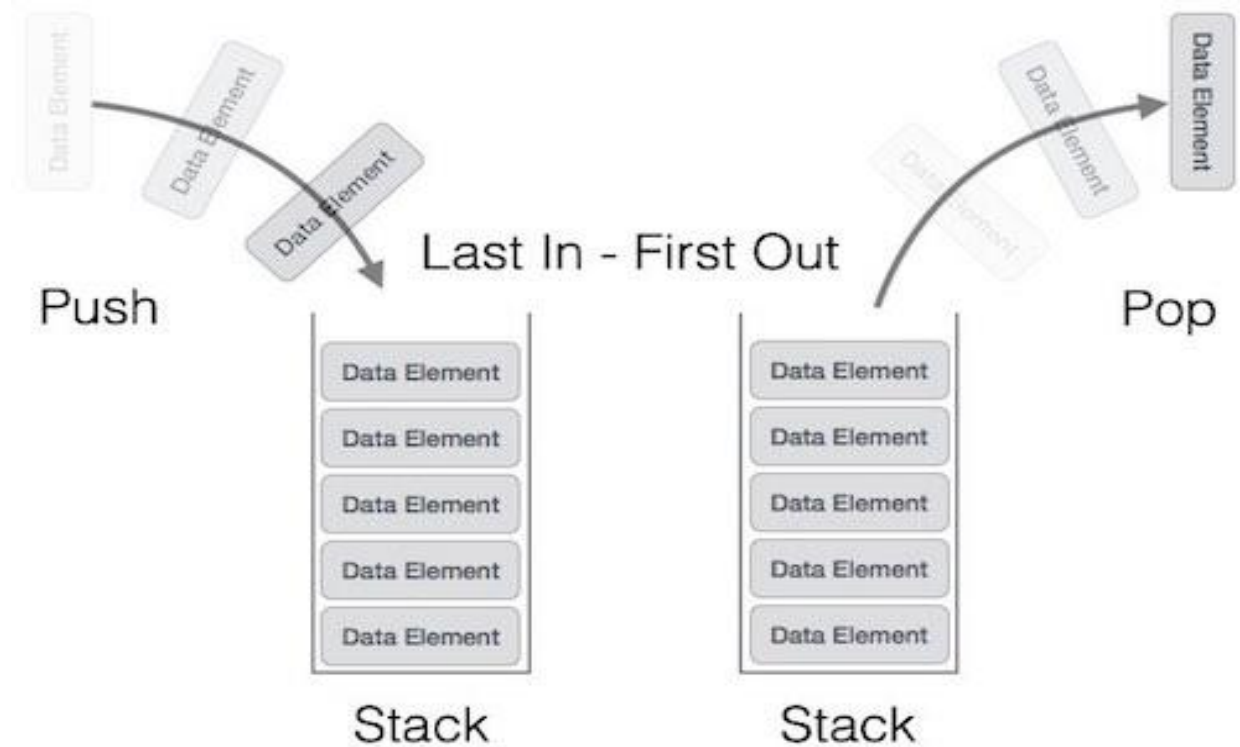
Stack

Basic Idea

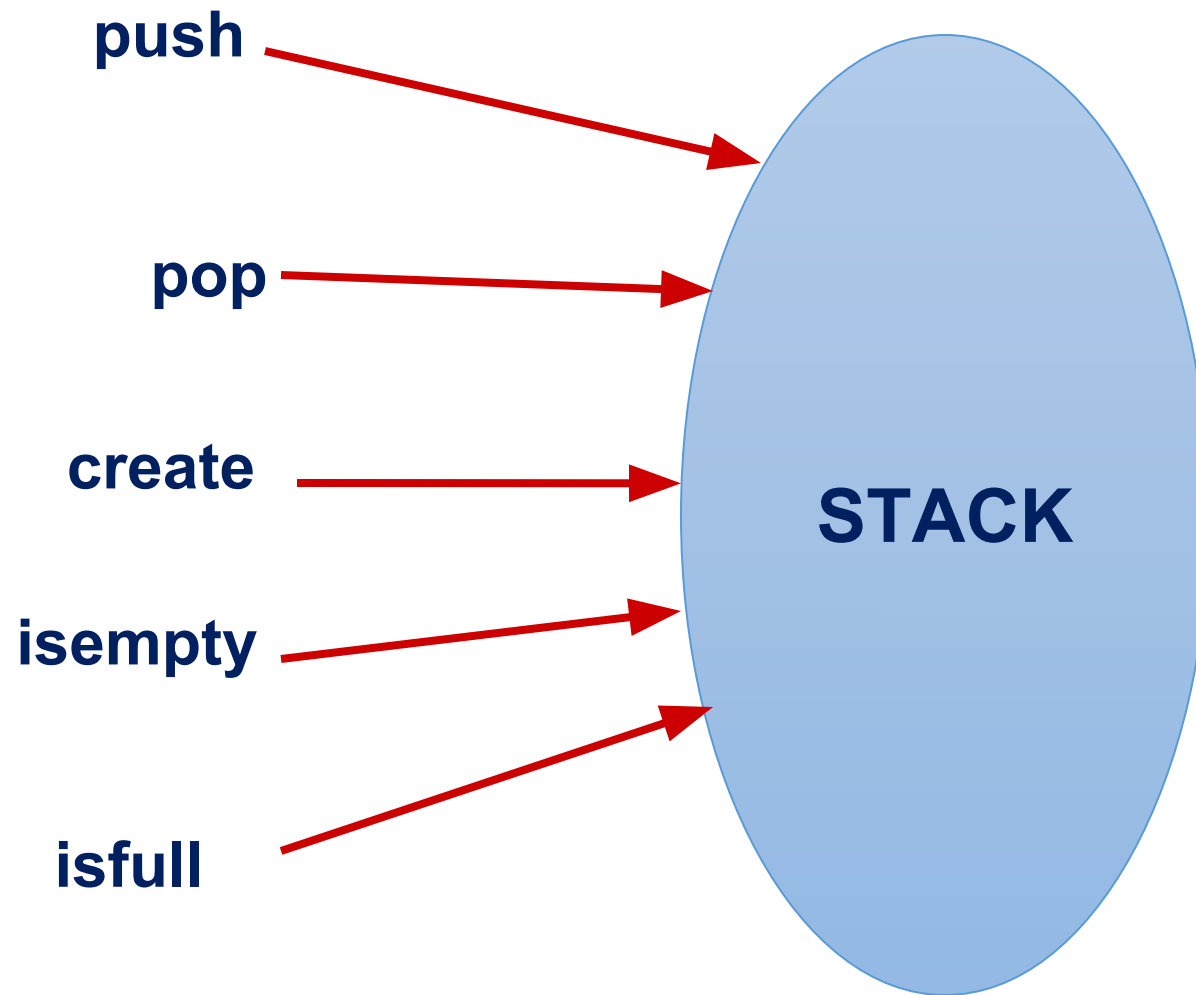
- A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.



Stack Representation



- Can be implemented by means of Array, Structure, Pointers and Linked List.
- Stack can either be a **fixed size or dynamic**.



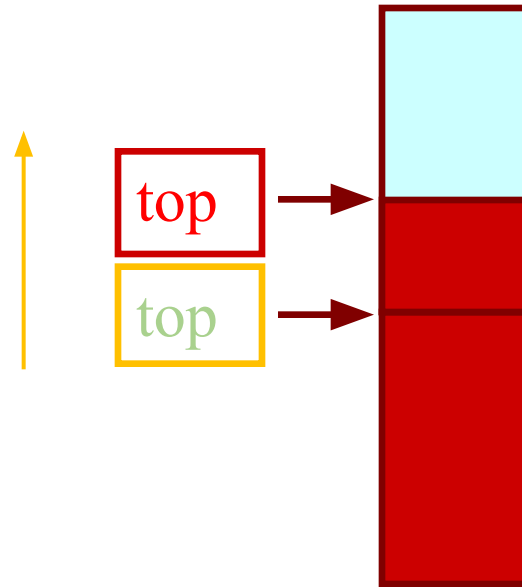
STACK: Last-In-First-Out (LIFO)

- **void push (stack *s, int element);**
/* Insert an element in the stack */
- **int pop (stack *s);**
/* Remove and return the top element */
- **void create (stack *s);**
/* Create a new stack */
- **int isempty (stack *s);**
/* Check if stack is empty */
- **int isfull (stack *s);**
/* Check if stack is full */

Assumption: **stack contains integer elements!**

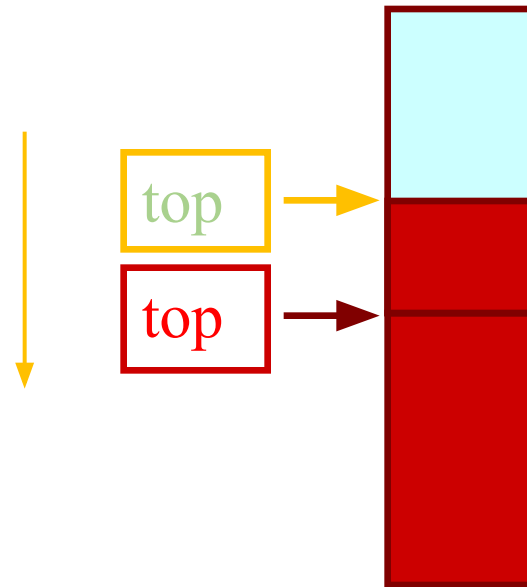
Stack using Array

Push using Stack



PUSH

Pop using Stack

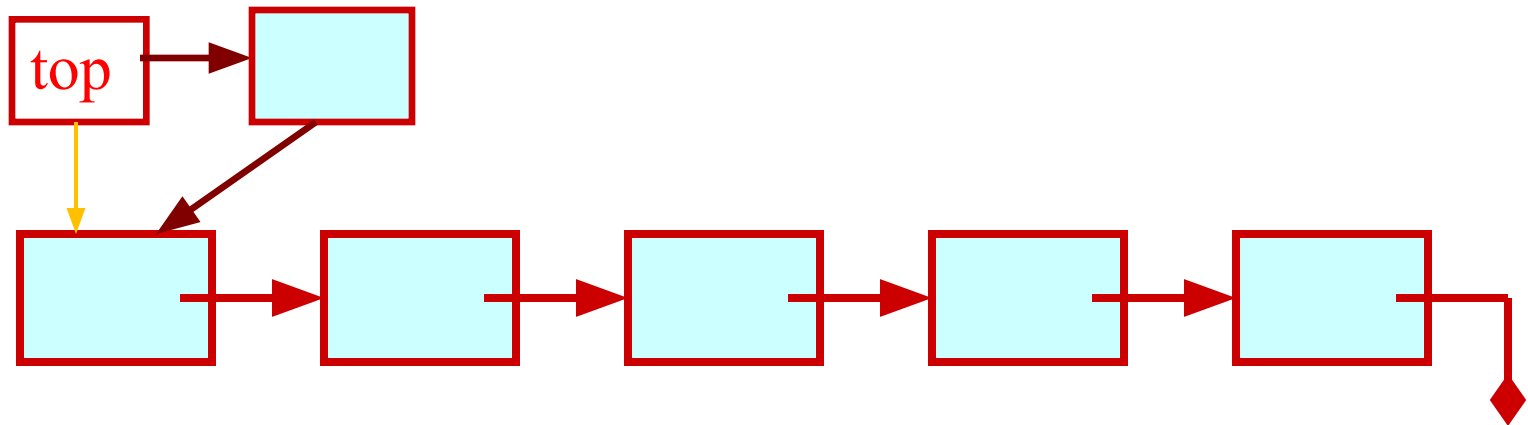


POP

Stack using Linked List

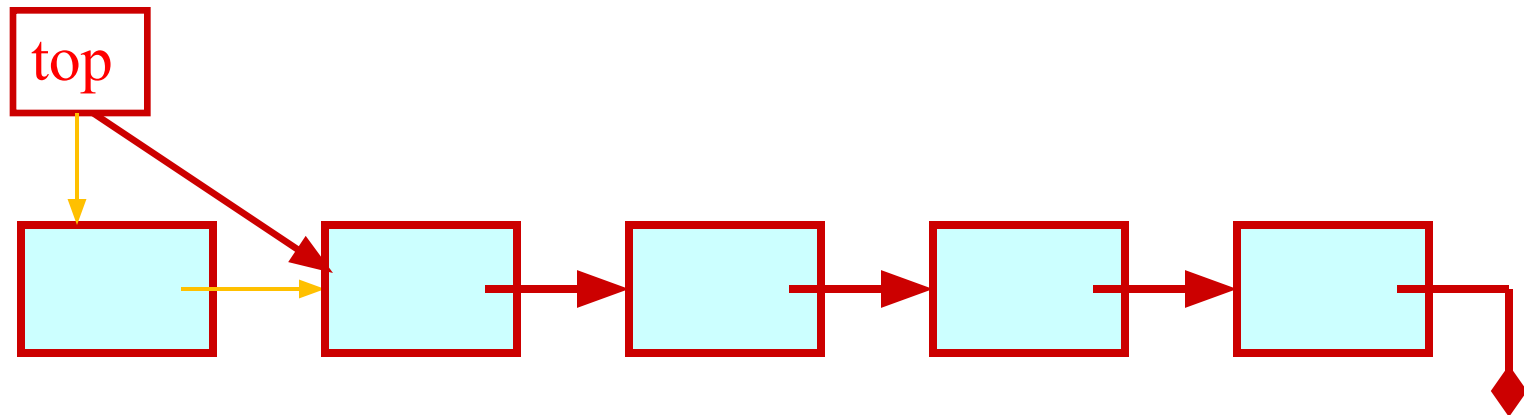
Push using Linked List

PUSH OPERATION



Pop using Linked List

POP OPERATION



Basic Idea

- In the array implementation, we would:
 - Declare an array of fixed size (which determines the maximum size of the stack).
 - Keep a variable which always points to the “top” of the stack.
 - Contains the array index of the “top” element.
- In the linked list implementation, we would:
 - Maintain the stack as a linked list.
 - A pointer variable top points to the start of the list.
 - The first element of the linked list is considered as the stack top.

Declaration

```
#define MAXSIZE 100

struct lifo
{
    int st[MAXSIZE];
    int top;
};
typedef struct lifo
    stack;
stack s;
```

ARRAY

```
struct lifo
{
    int value;
    struct lifo *next;
};
typedef struct lifo
    stack;

stack *top;
```

LINKED LIST

Stack Creation

```
void create (stack *s)
{
    s->top = -1;

    /* s->top points to
       last element
       pushed in;
       initially -1 */
}
```

ARRAY

```
void create (stack **top)
{
    *top = NULL;

    /* top points to NULL,
       indicating empty
       stack */
}
```

LINKED LIST

Pushing an element into stack

```
void push (stack *s, int element)
{
    if (s->top == (MAXSIZE-1))
    {
        printf ("\n Stack overflow");
        exit(-1);
    }
    else
    {
        s->top++;
        s->st[s->top] = element;
    }
}
```

ARRAY

```
void push (stack **top, int element)
{
    stack *new;

    new = (stack *)malloc (sizeof(stack));
    if (new == NULL)
    {
        printf ("\n Stack is full");
        exit(-1);
    }

    new->value = element;
    new->next = *top;
    *top = new;
}
```

LINKED LIST

Popping an element from stack

```
int pop (stack *s)
{
    if (s->top == -1)
    {
        printf ("\n Stack underflow");
        exit(-1);
    }
    else
    {
        return (s->st[s->top--]);
    }
}
```

ARRAY

```
int pop (stack **top)
{
    int t;
    stack *p;

    if (*top == NULL)
    {
        printf ("\n Stack is empty");
        exit(-1);
    }
    else
    {
        t = (*top)->value;
        p = *top;
        *top = (*top)->next;
        free (p);
        return t;
    }
}
```

LINKED LIST

Checking for stack empty

```
int isempty (stack *s)
{
    if (s->top == -1)
        return 1;
    else
        return (0);
}
```

ARRAY

```
int isempty (stack *top)
{
    if (top == NULL)
        return (1);
    else
        return (0);
}
```

LINKED LIST

Checking for Stack Full

```
int isempty (stack *s)
{
    if (s->top == -1)
        return 1;
    else
        return (0);
}
```

ARRAY

```
int isempty (stack *top)
{
    if (top == NULL)
        return (1);
    else
        return (0);
}
```

LINKED LIST

Example: A Stack using an Array

```
#include <stdio.h>
#define MAXSIZE 100

struct lifo
{
    int st[MAXSIZE];
    int top;
};
typedef struct lifo stack;
```

```
main() {
    stack A, B;
    create(&A);
    create(&B);
    push(&A, 10);
    push(&A, 20);
    push(&A, 30);
    push(&B, 100);
    push(&B, 5);

    printf ("%d %d", pop(&A),
pop(&B));

    push (&A, pop(&B));

    if (isempty(&B))
        printf ("\n B is empty");
    return;
}
```

Example: A Stack using Linked List

```
#include <stdio.h>
struct lifo
{
    int value;
    struct lifo *next;
};
typedef struct lifo stack;

main() {
    stack *A, *B;
    create(&A);
    create(&B);
    push(&A, 10);
    push(&A, 20);
    push(&A, 30);
    push(&B, 100);
    push(&B, 5);
```

```
    printf ("%d %d", pop(&A), pop(&B));

    push (&A, pop(&B));

    if (isempty(B))
        printf ("\n B is empty");
    return;
}
```

Applications of Stacks

- **Direct applications:**

- Page-visited history in a Web browser
- Undo sequence in a text editor
- Chain of method calls in the Java Virtual Machine
- Validate XML

- **Indirect applications:**

- Auxiliary data structure for algorithms
- Component of other data structures

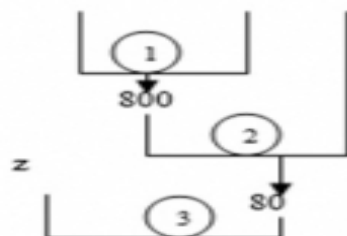
Expression Evaluation

- Expressions are evaluated by the 'C' compiler based on **precedence and associativity rules**.
- If an expression contains different **priority operators**, then the precedence rules are considered.

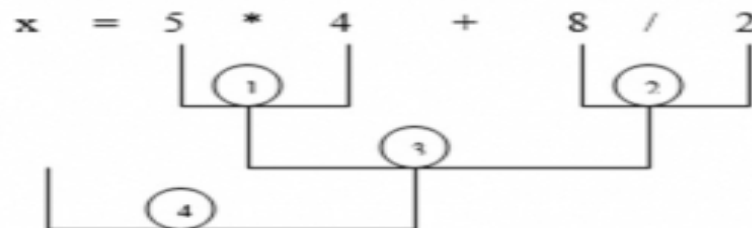
Eg: $z = a * b / c$

z = 40 * 20 / 10

Here '*' and '/' have same priority so, left to right associativity is considered



Eg: $x = 5 * 4 + 8 / 2$



$x = 24$

Parenthesis has highest priority and comma has least priority among operators

Expression Evaluation

Operators	Symbols
Parenthesis	(), { }, []
Exponents	$\wedge$
Multiplication and Division	$*$, $/$
Addition and Subtraction	$+$, $-$

Expression Evaluation

Operator Type	Operator	Associativity
Primary Expression Operators	() [] . -> <i>expr</i> ++ <i>expr</i> --	left-to-right
Unary Operators	* & + - ! ~ ++ <i>expr</i> -- <i>expr</i> (typecast) <u>sizeof</u>	right-to-left
Binary Operators	* / %	left-to-right
	+ -	

Expression Evaluation

	>> <<	
	< > <= >=	
	== !=	
	&	
	^	
	&&	
Ternary Operator	?:	right-to-left
Assignment Operators	= += -= *= /= %>= <<= &= ^= =	right-to-left

Infix, Prefix and Postfix Notations

INFIX NOTATION

- ★ **Infix notation** is the common arithmetic and logical formula notation, in which **operators are written** infix-style **between the operands** they act on

- ★ E.g. **A + B**

<operand> <operator> <operand>

Infix, Prefix and Postfix Notations

POSTFIX NOTATION

- ★ In Postfix notation, the **operator** comes **after the Operand**.
- ★ For example, the Infix expression **A+B** will be written as **AB+** in its **Postfix Notation**.
- ★ Postfix is also called '**Reverse Polish Notation**'

<operand> <operand> <operator>

Infix, Prefix and Postfix Notations

PREFIX NOTATION

- ★ In Prefix notation, the **operator** comes **before the operand**.
- ★ The Infix expression **A+B** will be written as **+AB** in its Prefix Notation.
- ★ Prefix is also called '**Polish Notation**'

<operator> <operand> <operand>

Infix, Prefix and Postfix Notations

- Infix: operators placed between operands:

$$A+B*C$$

- Postfix: operands appear before their operators:-

$$ABC*+$$

- Prefix: operands appear after their operators:-

$$+A*BC$$

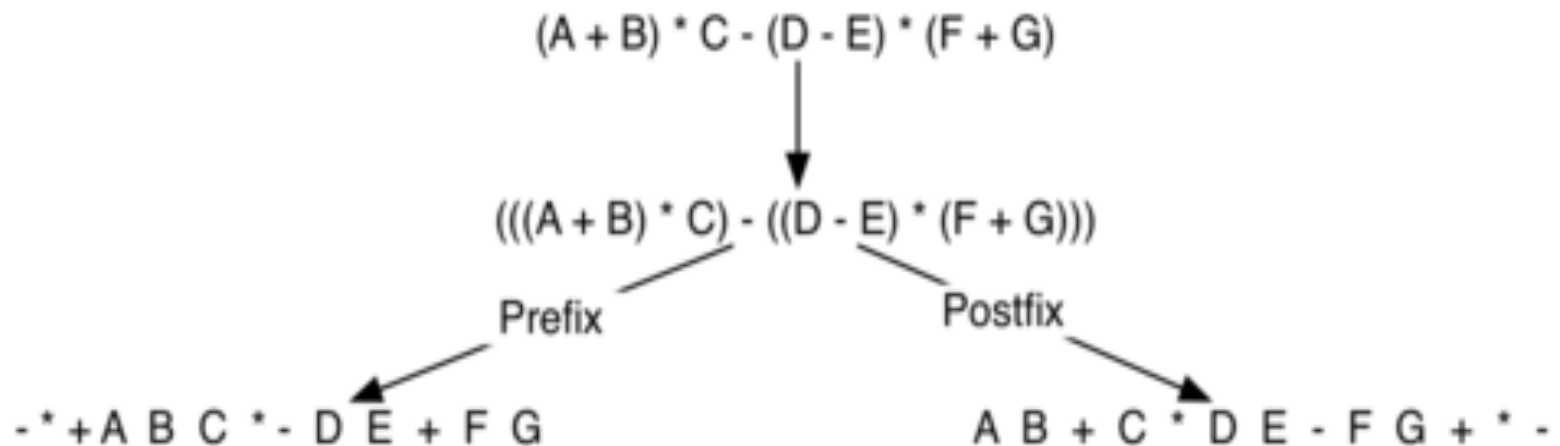
Infix Expression	Prefix Expression	Postfix Expression
$A + B$	$+ A B$	$A B +$
$A + B * C$	$+ A * B C$	$A B C * +$

- There are no precedence rules to learn in postfix notation, and parentheses are never needed

Infix, Prefix and Postfix Expressions

Infix Expression	Prefix Expression	Postfix Expression
$(A + B) * C$	$* + A B C$	$A B + C *$
$A + B * C + D$	$++ A * B C D$	$A B C * + D +$
$(A + B) * (C + D)$	$* + A B + C D$	$A B + C D + *$
$A * B + C * D$	$+ * A B * C D$	$A B * C D * +$
$A + B + C + D$	$+++ A B C D$	$A B + C + D +$

Converting a Complex Expression to Prefix and Postfix Notations



Infix to Postfix

Infix	Postfix
$A + B$	$A B +$
$A + B * C$	$A B C * +$
$(A + B) * C$	$A B + C *$
$A + B * C + D$	$A B C * + D +$
$(A + B) * (C + D)$	$A B + C D + *$
$A * B + C * D$	$A B * C D * +$

$A + B * C \quad \square \quad (A + (B * C)) \quad \square \quad (A + (B C *)) \quad \square \quad A B C * +$

$A + B * C + D \quad \square \quad ((A + (B * C)) + D) \quad \square \quad ((A + (B C *)) + D) \quad \square \quad ((A B C * +) + D) \quad \square \quad A B C * + D +$

Infix to postfix conversion

- Use a stack for processing operators (push and pop operations).
- Scan the sequence of operators and operands from left to right and perform one of the following:
 - output the operand,
 - push an operator of higher precedence,
 - pop an operator and output, till the stack top contains operator of a lower precedence and push the present operator.

The algorithm steps (Infix to postfix conversion)

1. Print operands as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
3. If the incoming symbol is a left parenthesis, push it on the stack.
4. If the incoming symbol is a right parenthesis, pop the stack and print the operators until you see a left parenthesis. Discard the pair of parentheses.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.

The algorithm steps (Infix to postfix conversion)

6. If the incoming symbol has equal precedence with the top of the stack, use association. If the association is left to right, pop and print the top of the stack and then push the incoming operator. If the association is right to left, push the incoming operator.
7. If the incoming symbol has lower precedence than the symbol on the top of the stack, pop the stack and print the top operator. Then test the incoming operator against the new top of stack.
8. At the end of the expression, pop and print all operators on the stack. (No parentheses should remain.)

The algorithm steps (Infix to prefix conversion)

1. First, reverse the infix expression given in the problem.
2. Scan the expression from left to right.
3. Whenever the operands arrive, print them.
4. If the operator arrives and the stack is found to be empty, then simply push the operator into the stack.
5. If the incoming operator has higher precedence than the TOP of the stack, push the incoming operator into the stack.
6. If the incoming operator has the same precedence with a TOP of the stack, push the incoming operator into the stack.
7. If the incoming operator has lower precedence than the TOP of the stack, pop, and print the top of the stack. Test the incoming operator against the top of the stack again and pop the operator from the stack till it finds the operator of a lower precedence or same precedence.

The algorithm steps (Infix to prefix conversion)

8. If the incoming operator has the same precedence with the top of the stack and the incoming operator is '^', then pop the top of the stack till the condition is true. If the condition is not true, push the '^' operator.
9. When we reach the end of the expression, pop, and print all the operators from the top of the stack.
10. If the operator is ')', then push it into the stack.
11. If the operator is '(', then pop all the operators from the stack till it finds) opening bracket in the stack.
12. If the top of the stack is ')', push the operator on the stack.
13. At the end, reverse the output.

Infix to Postfix Conversion

Requires operator precedence information

Operands:

Add to postfix expression.

Close parenthesis:

pop stack symbols until an open parenthesis appears.

Operators:

Pop all stack symbols until a symbol of lower precedence appears. Then push the operator.

End of input:

Pop all remaining stack symbols and add to the expression.

Infix to Postfix Rules

Expression:

A * (B + C * D) + E

becomes

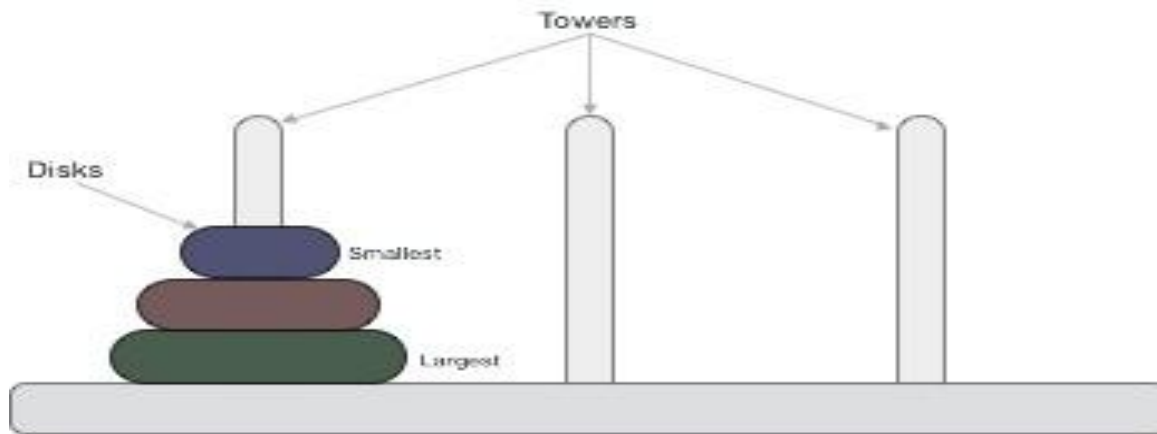
A B C D * + * E +

Postfix notation is
also called as
Reverse Polish
Notation (RPN)

	Current symbol	Operator Stack	Postfix string
1	A		A
2	*	*	A
3	(	* (	A
4	B	* (	A B
5	+	* (+	A B
6	C	* (+	A B C
7	*	* (+ *	A B C
8	D	* (+ *	A B C D
9	)	*	A B C D * +
10	+	+	A B C D * + *
11	E	+	A B C D * + * E
12			A B C D * + * E +

Tower of Hanoi

- Tower of Hanoi, is a **mathematical puzzle** which consists of three towers (pegs) and more than one rings is as depicted –



- These rings are of different sizes and stacked upon in an ascending order, i.e. the smaller one sits over the larger one. There are other variations of the puzzle where the number of disks increase, but the tower count remains the same.

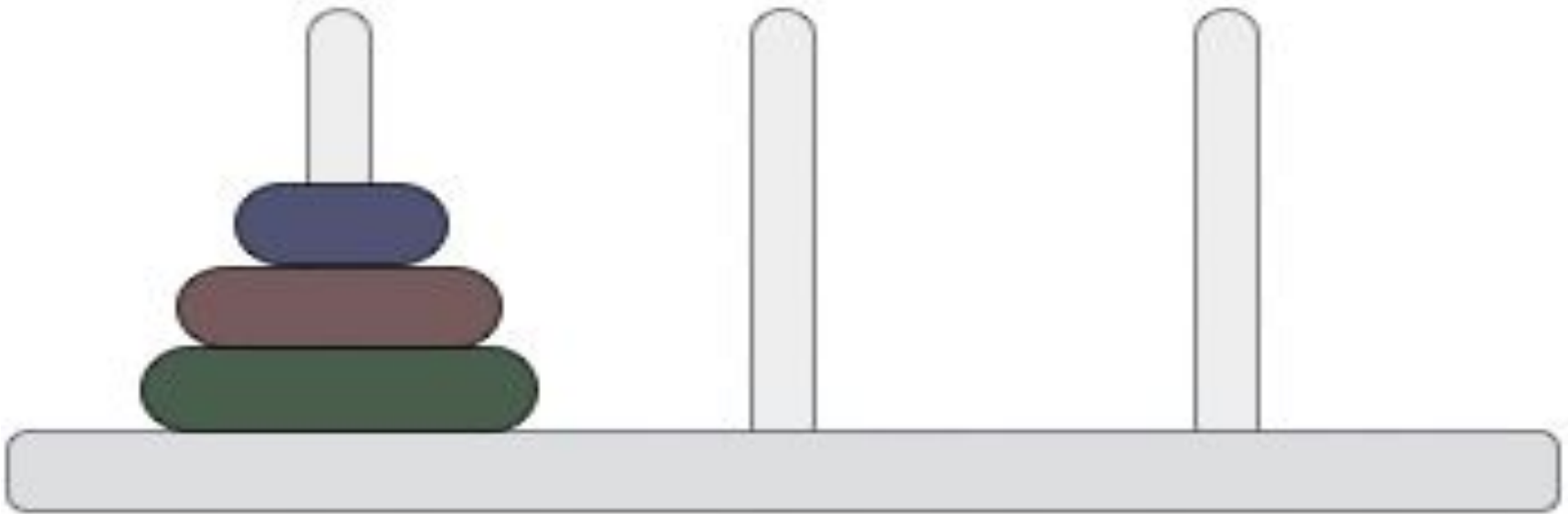
Tower of Hanoi

Rules

- The mission is to move all the disks to some another tower without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi are –
 - Only one disk can be moved among the towers at any given time.
 - Only the "top" disk can be removed.
 - No large disk can sit over a small disk.
- Following is an animated representation of solving a Tower of Hanoi puzzle with three disks.
- Tower of Hanoi puzzle with n disks can be solved in minimum $2^n - 1$ steps. This presentation shows that a puzzle with 3 disks has taken $2^3 - 1 = 7$ steps.

Tower of Hanoi

Step: 0



Tower of Hanoi

Algorithm

- To write an algorithm for Tower of Hanoi, first we need to learn how to solve this problem with lesser amount of disks, say $\rightarrow$ 1 or 2. We mark three towers with name, source, destination and aux (only to help moving the disks). If we have only one disk, then it can easily be moved from source to destination peg.
- If we have 2 disks –
- First, we move the smaller (top) disk to aux peg.
- Then, we move the larger (bottom) disk to destination peg.
- And finally, we move the smaller disk from aux to destination peg.

Tower of Hanoi

Step: 0



So now, we are in a position to design an algorithm for Tower of Hanoi with more than two disks. We divide the stack of disks in two parts. The largest disk (n^{th} disk) is in one part and all other ($n-1$) disks are in the second part.

Tower of Hanoi

Our ultimate aim is to move disk n from source to destination and then put all other ($n-1$) disks onto it. We can imagine to apply the same in a recursive way for all given set of disks.

The steps to follow are –

Step 1 – Move $n-1$ disks from source to aux

Step 2 – Move n th disk from source to dest

Step 3 – Move $n-1$ disks from aux to dest

Tower of Hanoi

A recursive algorithm for Tower of Hanoi can be driven as follows –

START

Procedure Hanoi(disk, source, dest, aux)

IF disk == 1, THEN

 move disk from source to dest

ELSE

 Hanoi(disk - 1, source, aux, dest) // Step 1

 move disk from source to dest // Step 2

 Hanoi(disk - 1, aux, dest, source) // Step 3

END IF

END Procedure

STOP

Stack ADT

- In Stack ADT Implementation instead of data being stored in each node, the pointer to data is stored. The program allocates memory for the data and address is passed to the stack ADT. The head node and the data nodes are encapsulated in the ADT. The calling function can only see the pointer to the stack. The stack head structure also contains a pointer to top and count of number of entries currently in stack.

50
40
30
20
10

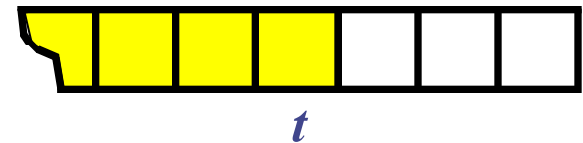
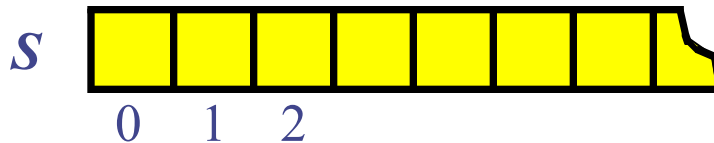
- push() – Insert an element at one end of the stack called top.
- pop() – Remove and return the element at the top of the stack, if it is not empty.
- peek() – Return the element at the top of the stack without removing it, if the stack is not empty.
- size() – Return the number of elements in the stack.
- isEmpty() – Return true if the stack is empty, otherwise return false.
- isFull() – Return true if the stack is full, otherwise return false.

Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

```
Algorithm size()  
    return t + 1
```

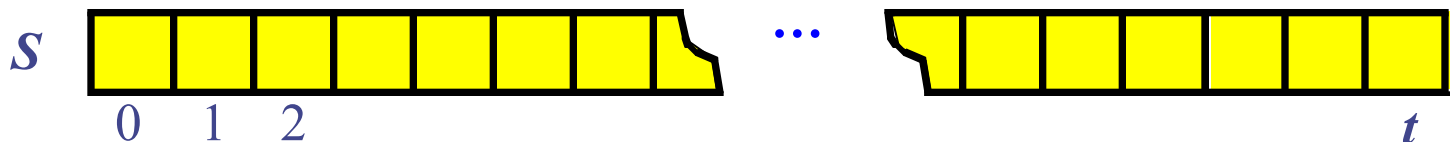
```
Algorithm pop()  
    if empty() then  
        throw EmptyStackException  
    else  
        t = t - 1  
        return S[t + 1]
```



Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then throw a FullStackException
 - Limitation of the array-based implementation
 - Not intrinsic to the Stack ADT

```
Algorithm push(o)
    if  $t = S.length - 1$  then
        throw FullStackException
    else
         $t = t + 1$ 
         $S[t] = o$ 
```



Performance and Limitations

- array-based implementation of stack ADT

Performance

- Let n be the number of elements in the stack
- The space used is $O(n)$
- Each operation runs in time $O(1)$

Limitations

- The maximum size of the stack must be defined a priori, and cannot be changed
- Trying to push a new element into a full stack causes an implementation-specific exception

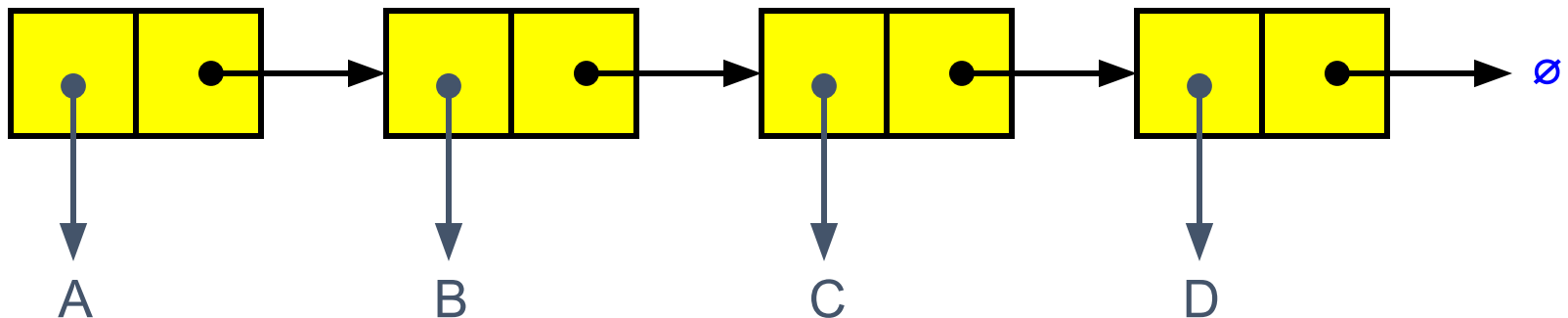
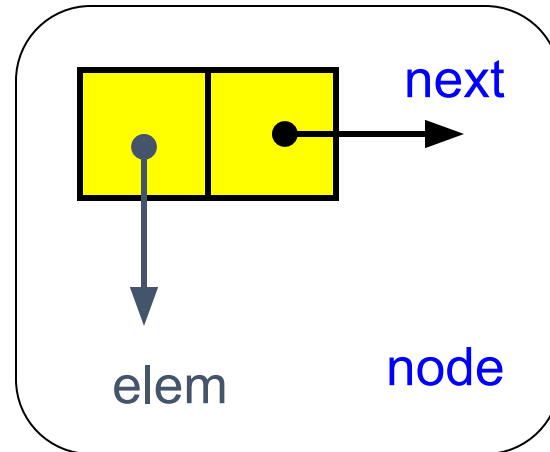
Array-based Stack in C++

```
template <typename Object>
class ArrayStack {
private:
    int capacity;           // stack
    capacity
    Object *S;              //
    stack array
    int top;                // top
    of stack
public:
    ArrayStack(int c) {
        capacity = c;
        S = new Object[capacity];
        t = -1;
    }
}
```

```
    bool isEmpty()
    { return (t < 0); }
    Object pop()
    {
        if(isEmpty())
            throw
            EmptyStackException
            ("Access to empty
            stack");
        return S[t--];
    }
// ... (other functions omitted)
```

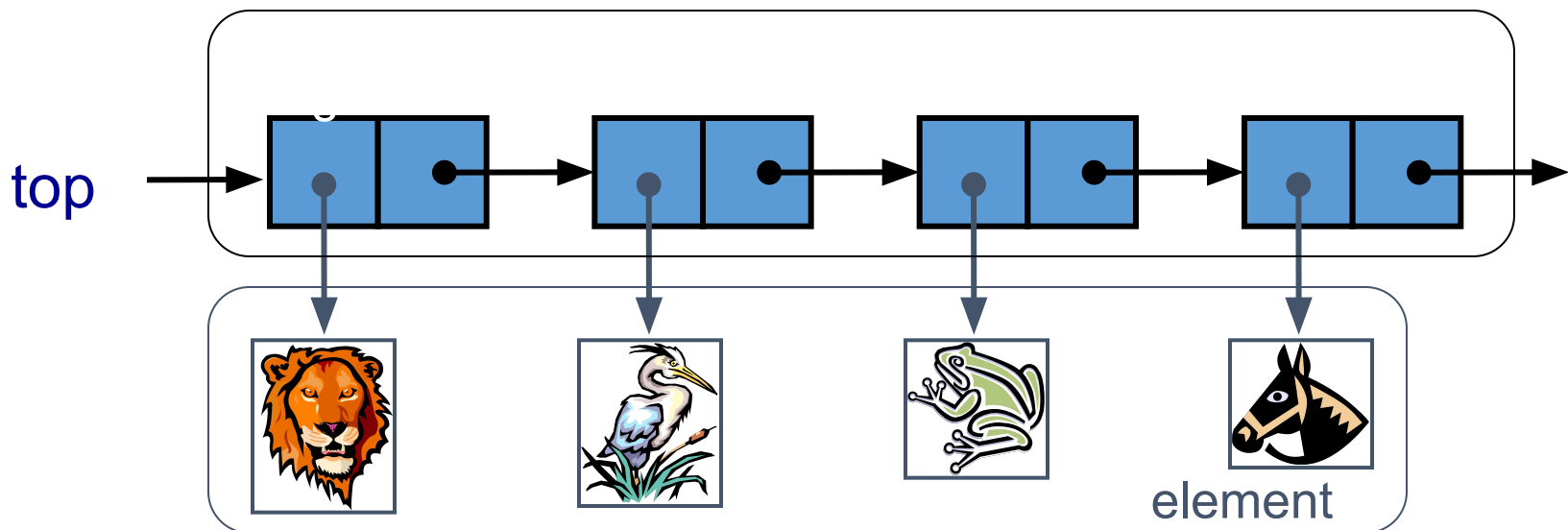
Singly Linked List

- A singly linked list is a concrete data structure consisting of a sequence of nodes
- Each node stores
 - element
 - link to the next node



Stack with a Singly Linked List

- We can implement a stack with a singly linked list
- The top element is stored at the first node of the list
- The space used is $O(n)$ and each operation of the Stack ADT takes $O(1)$ time



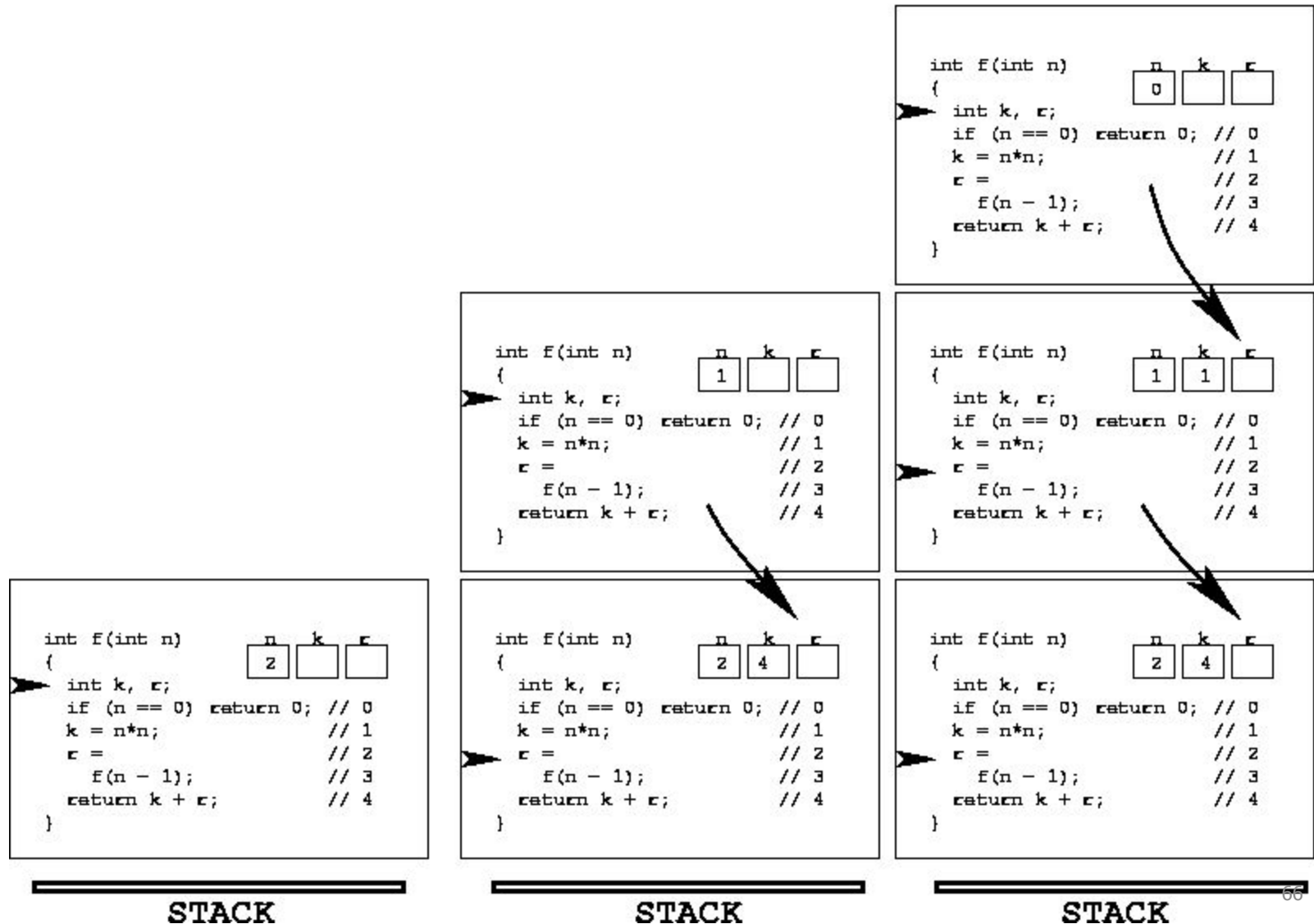
Recursion Using Stacks

What's Going On?

```
int main() {  
    // Get Input  
    int x, y;  
    cout << "Enter value x: ";  
    cin >> x;  
  
    // Call f  
    y = f(x);  
  
    // Print results  
    cout << y << endl;  
    return 0;  
}
```

```
#include <iostream>  
using namespace std;  
  
int f(int n) {  
    int k, r;  
    if (n == 0)  
        return 0;           // 0  
    k = n*n;  
    // 1  
    r =  
        // 2  
        f(n - 1);  
    // 3  
    return k + r;           // 4  
}
```

What's Going On...



Loop

- Then loop as long as there are activation records on stack
- At each iteration of loop we simulate execution of next line of activation record on top of stack

Queue

Basic Idea

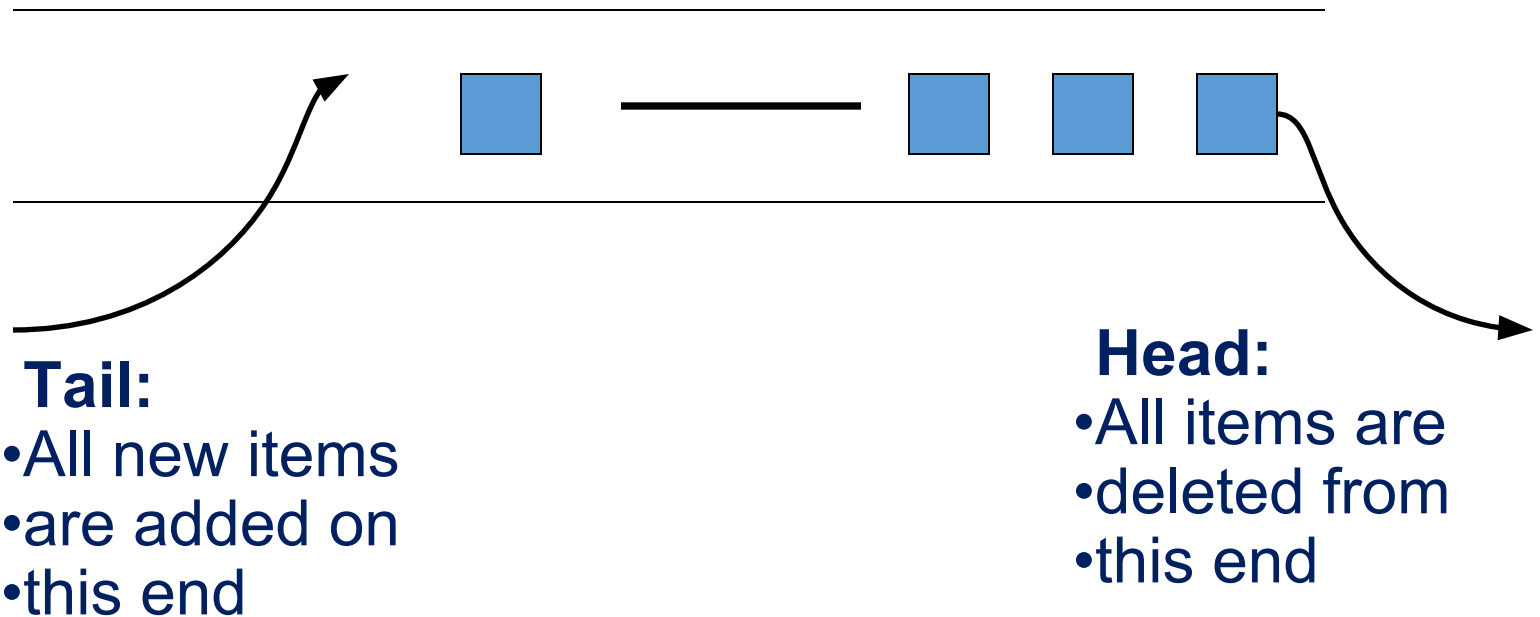
- Queue is an abstract data structure, somewhat similar to Stacks. Unlike stacks, a queue is **open at both its ends**. One end is always used to **insert data (enqueue)** and the other is used to **remove data (dequeue)**.



Definition of a Queue

- A queue is a data structure that models/enforces the **first-come first-serve order**, or equivalently the first-in first-out (**FIFO**) order.
- That is, the element that is inserted first into the queue will be the element that will be deleted first, and the element that is inserted last is deleted last.
- A waiting line is a good real-life example of a queue. (In fact, the British word for “line” is “queue”.)

A Graphic Model of a Queue



Queue Representation



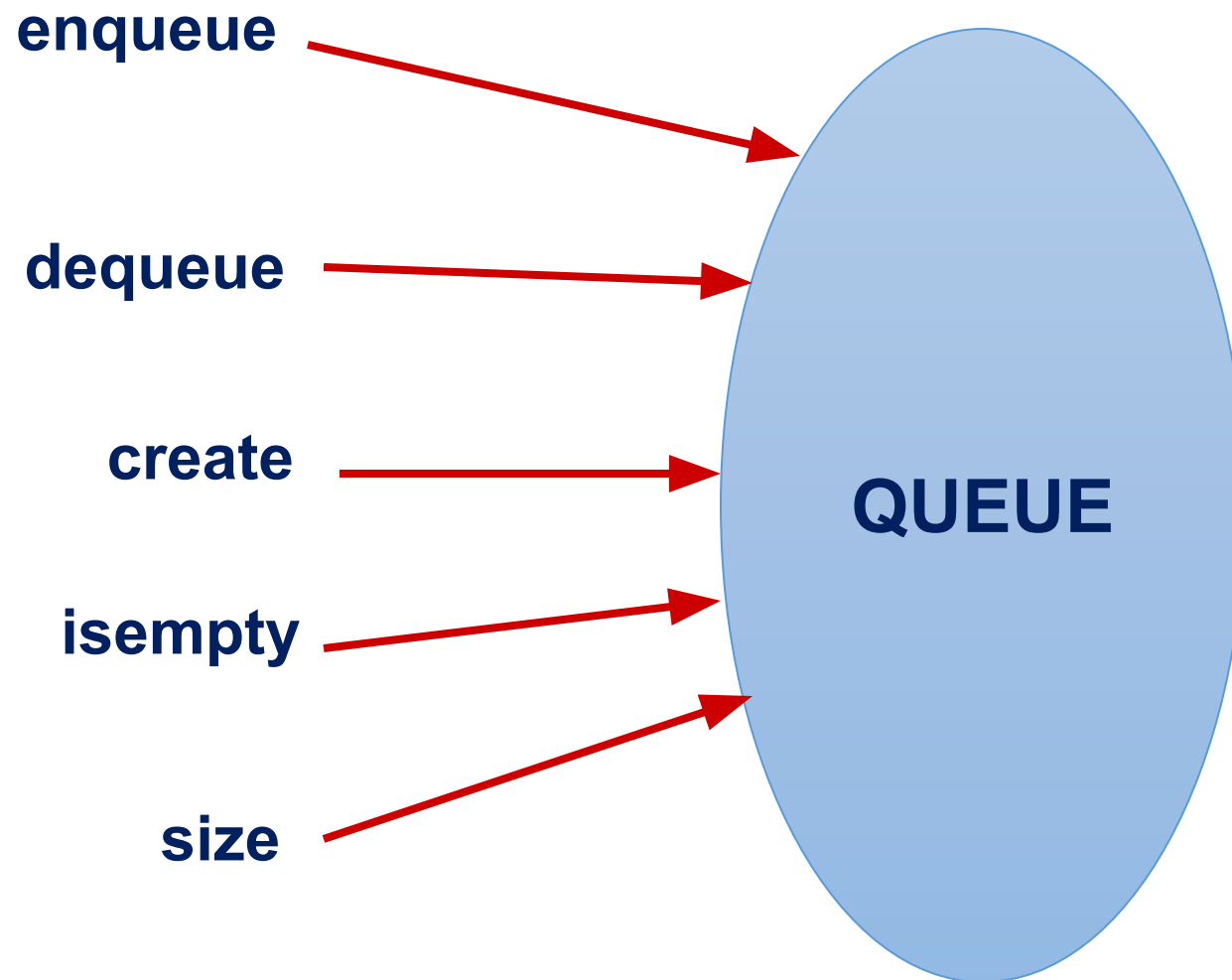
- As in stacks, a queue can also be implemented using **Arrays, Linked-lists, Pointers and Structures.**

Operations on Queues

- Insert(item): (also called enqueue)
 - It adds a new item to the tail of the queue
- Remove(): (also called delete or dequeue)
 - It deletes the head item of the queue, and returns to the caller. If the queue is already empty, this operation returns NULL
- getHead():
 - Returns the value in the head element of the queue
- getTail():
 - Returns the value in the tail element of the queue
- isEmpty()
 - Returns true if the queue has no items
- size()
 - Returns the number of items in the queue

Examples of Queues

- An electronic mailbox is a queue
 - The ordering is chronological (by arrival time)
- A waiting line in a store, at a service counter, on a one-lane road
- Equal-priority processes waiting to run on a processor in a computer system



QUEUE: First-In-First-Out (LIFO)

```
void enqueue (queue *q, int element);
/* Insert an element in the queue */
int dequeue (queue *q);
/* Remove an element from the queue */
queue *create();
/* Create a new queue */
int isempty (queue *q);
/* Check if queue is empty */
int size (queue *q);
/* Return the no. of elements in queue */
```

Assumption: queue contains integer elements!

Queue ADT

10	20	30	40	50	60
----	----	----	----	----	----

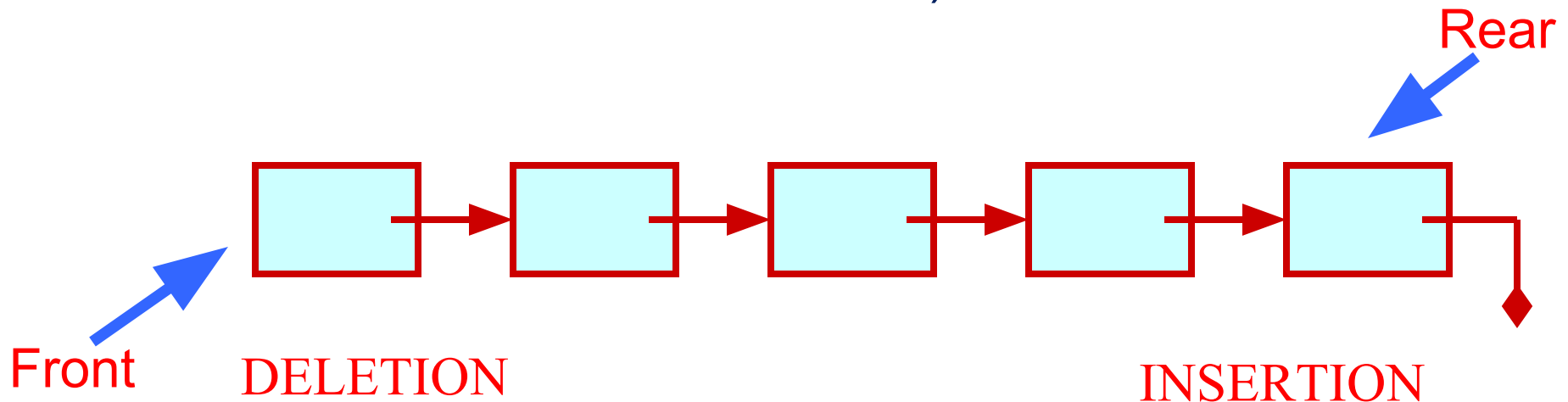
- The queue abstract data type (ADT) follows the basic design of the stack abstract data type. Each node contains a void pointer to the data and the link pointer to the next element in the queue. The program's responsibility is to allocate memory for storing the data.
 - enqueue() – Insert an element at the end of the queue.
 - dequeue() – Remove and return the first element of the queue, if the queue is not empty.
 - peek() – Return the element of the queue without removing it, if the queue is not empty.
 - size() – Return the number of elements in the queue.
 - isEmpty() – Return true if the queue is empty, otherwise return false.
 - isFull() – Return true if the queue is full, otherwise return false.

Queue using Linked List

Basic Idea

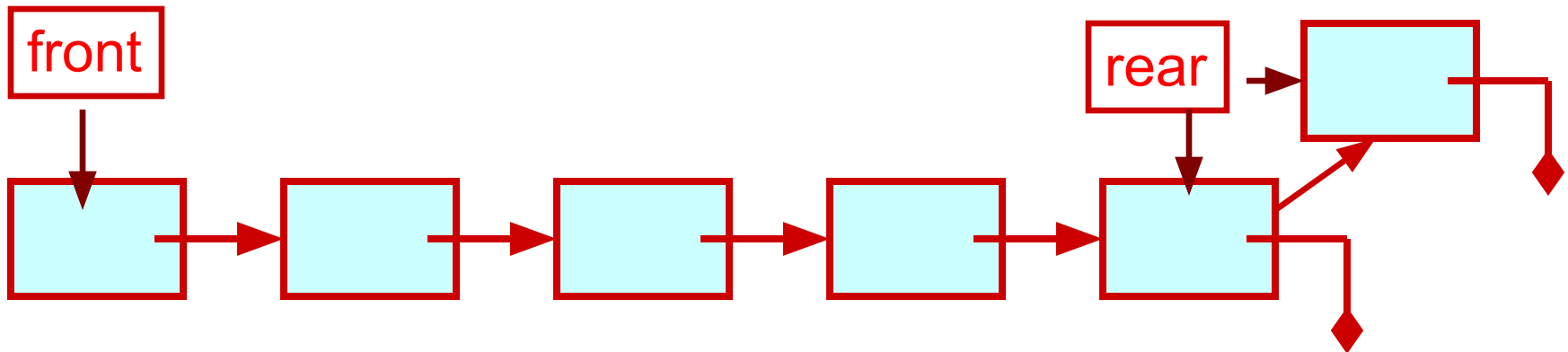
- **Basic idea:**

- Create a linked list to which items would be added to one end and deleted from the other end.
- Two pointers will be maintained:
 - One pointing to the beginning of the list (point from where elements will be deleted).
 - Another pointing to the end of the list (point where new elements will be inserted).



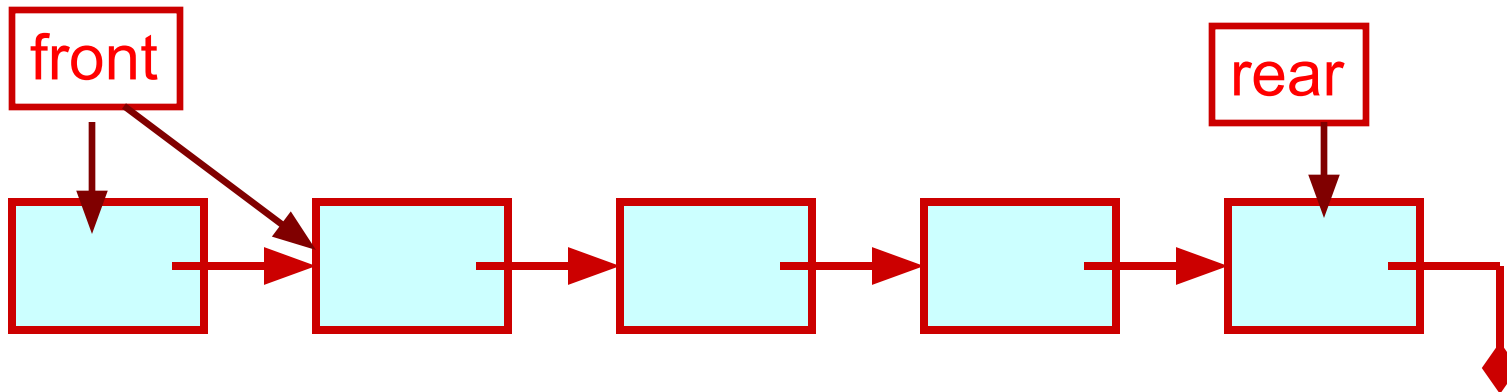
Queue: Linked List Structure

ENQUEUE



Queue: Linked List Structure

DEQUEUE



Example :Queue using Linked List

```
struct qnode
{
    int val;
    struct qnode *next;
};

struct queue
{
    struct qnode *qfront, *qrear;
};
typedef struct queue QUEUE;
```

```
void enqueue (QUEUE *q,int element)
{
    struct qnode *q1;
    q1=(struct qnode *)malloc(sizeof(struct qnode));
    q1->val= element;
    q1->next=q->qfront;
    q->qfront=q1;
}
```

Example :Queue using Linked List

```
int size (queue *q)
{
    queue *q1;
    int count=0;
    q1=q;
    while(q1!=NULL)
    {
        q1=q1->next;
        count++;
    }
    return count;
}
```

```
int peek (queue *q)
{
    queue *q1;
    q1=q;

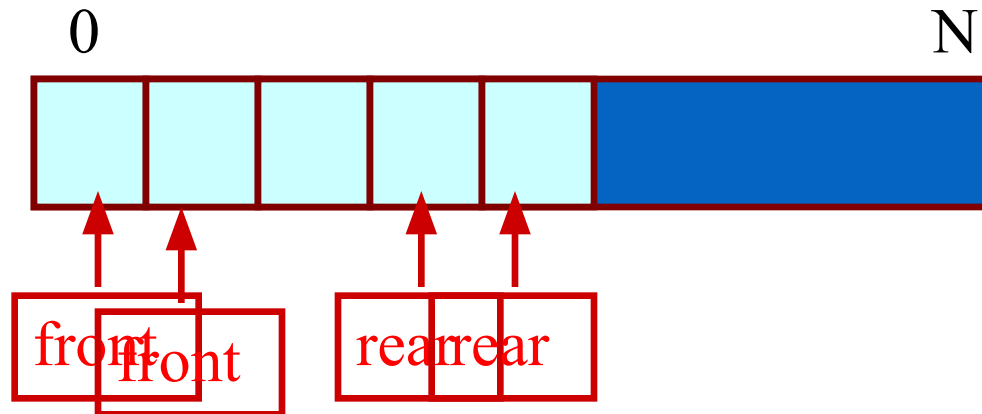
    while(q1->next!=NULL)
        q1=q1->next;
    return (q1->val);
}
```

```
int dequeue (queue *q)
{
    int val;
    queue *q1,*prev;
    q1=q;
    while(q1->next!=NULL)
    {
        prev=q1;
        q1=q1->next;
    }
    val=q1->val;
    prev->next=NULL;
    free(q1);
    return (val);
}
```

Problem With Array Implementation

- The size of the queue depends on the number and order of enqueue and dequeue.
- It may be situation where memory is available but enqueue is not possible.

ENQUEUE DEQUEUE
Effective queuing storage area of array gets reduced.



Use of circular array indexing

Applications of Queues

- **Direct applications:-**

- Waiting lists
- Access to shared resources (e.g., printer)
- Multiprogramming

- **Indirect applications:-**

- Auxiliary data structure for algorithms
- Component of other data structures

Queue as a Class

- Much like stacks and linked lists were designed and implemented as classes, a queue can be conveniently packaged as a class
- It seems natural to think of a queue as similar to a linked list, but with more basic operations, to enforce the FIFO order of insertion and deletion

A Rough Class for Queue

```
class Queue{
    public:
        typedef int datatype; // datatype is the type of items to be
        //added to the queue. By changing int to some other
        // type, the
        // queue is easily changed to handle other data types
        Queue( );
        void enqueue(datatype x);
        datatype dequeue( );
        datatype peekHead( );
        datatype peekTail( );
        int size( );
        bool isEmpty( );

    private:
        //A container for items. It's determined in the implementation
};
```

A Class for Queue: A Linked List Implementation

```
class Queue{
    public:
        typedef int datatype;
        Queue( ) { };
        void enqueue(datatype x) {q.insertTail(x);};
        datatype peekHead( ) {assert(size()>0); q.getHead( )->getData(
);};
        datatype peekTail( ) {assert(size()>0); q.getTail( )->getData( );};
        datatype dequeue( ) {assert(size()>0); int x=peekHead( );
                        q.removeHead( ); return x;};
        int size( ) {return q.size( );};
        bool isEmpty( ) {return q.isEmpty( );};

    private:
        List q;
};    // Time: All member functions take O(1) time.
```


A Dynamic-Array Implementation of the Queue Class

- The container for items is a dynamic array
- It is accomplished as:
`datatype *p=new datatype[50];`

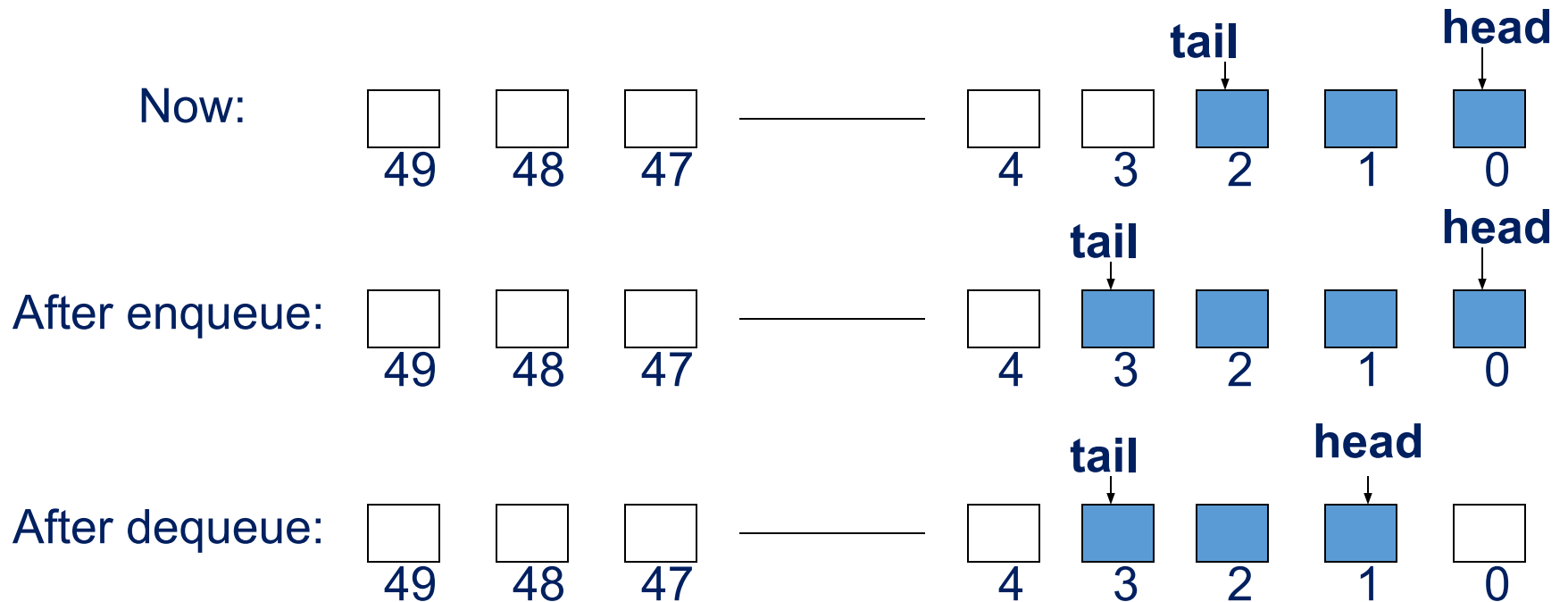
A Class for Queue using Dynamic Arrays

```
class Queue{
    public:
        typedef int datatype;
        Queue(int capacity = 50) ;
        void enqueue(datatype x);    datatype dequeue( );
        datatype peekHead( );        datatype peekTail( );
        int size( );                  bool isEmpty( );

    private:
        int capacity; // default value is 50
        datatype *p; // pointer a dynamic array created by constructor
        int length; // number of actual elements in the queue
        int head; // index of the head element in the array
        int tail; // index of the tail element in the array
};
```

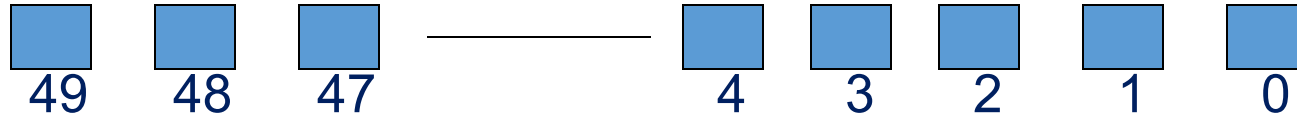
How head and tail Change

- **head** increases by 1 after each dequeue()
- **tail** increases by 1 after each enqueue()

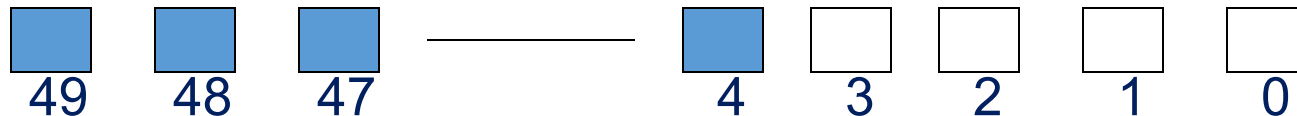


False-Overflow Issue First

- Suppose 50 calls to enqueue have been made, so now the queue array is full



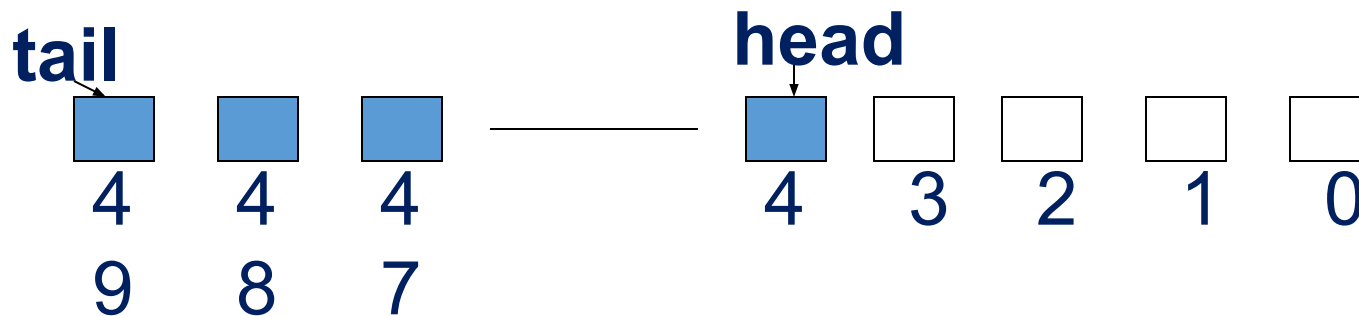
- Assume 4 calls to dequeue() are made



- Assume a call to enqueue() is made now. The tail part seems to have no space, but the front has 4 unused spaces; if never used, they are wasted.

Solution: A Circular Queue

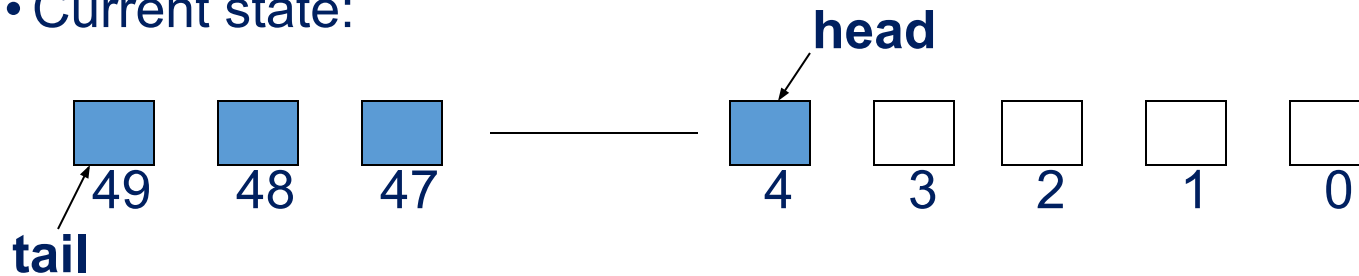
- Allow the head (and the tail) to be moving targets
- When the tail end fills up and front part of the array has empty slots, new insertions should go into the front end



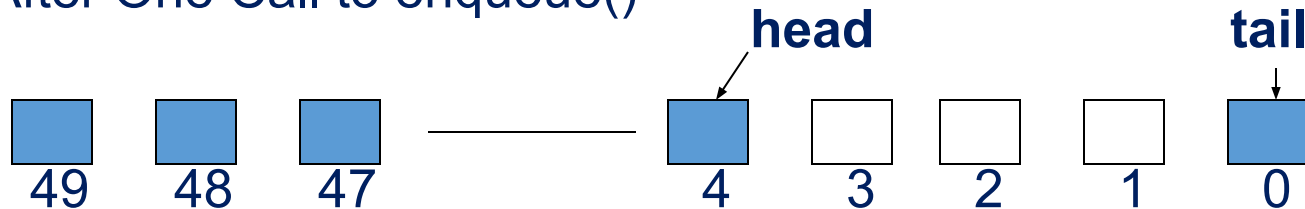
- Next insertion goes into slot 0, and tail tracks it. The insertion after that goes into a lot 1, etc.

Illustration of Circular Queues

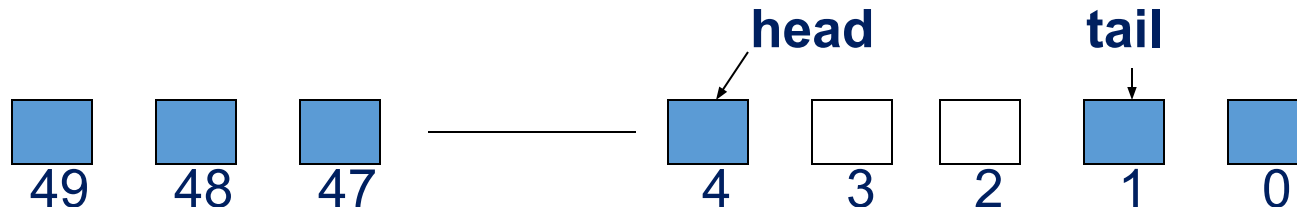
- Current state:



- After One Call to enqueue()



- After One Call to enqueue()



Numerics for Circular Queues

- **head** increases by (1 modulo capacity) after each `dequeue( )`:

head = (**head** + 1) % capacity;

- **tail** increases by (1 modulo capacity) after each `enqueue( )`:

tail = (**tail** + 1) % capacity;

Complete Implementation of Queues with Dynamic Arrays

- Show the class structure as a refresher (next slide)
- The implementations of the constructor and member functions will follow afterwards

The Class Structure (revisited)

- **class** Queue{

- public:**

- typedef int** *datatype*;

- Queue(**int** capacity = 50) ;

- void** enqueue(*datatype* x); *datatype* dequeue();

- datatype* peekHead(); *datatype* peekTail();

- int** size(); **bool** isEmpty();

- private:**

- int** capacity; // default value is 50

- datatype* *p; // pointer a dynamic array created by constructor

- int** length; // number of actual elements in the queue

- int head**; // index of the head element in the array

- int tail**; // index of the tail element in the array

- };

Implementations of the Members

```
Queue::Queue(int capacity){  
    this.capacity = capacity;  
    p = new datatype[capacity];  
    length=0; head=0; tail=0;  
};  
  
int Queue::size( ){return length;};  
  
bool Queue::isEmpty( ) {return (length==0);};  
  
Queue::datatype Queue::peekHead( ){assert(length>0);return  
    p[head];};  
  
Queue::datatype Queue::peekTail( ){assert(length>0); return  
    p[tail];};
```

Time: All $O(1)$ time.

Implementation enqueue()

```
void Queue::enqueue(datatype x){  
    if (length==0) {p[tail]=x;   length++;}  
    else if (length<capacity) {length++;   tail=tail+1 % capacity;   p[tail]=x; }  
    else {// Filled. Create a new array twice big. Copy current array to it. Add x.  
        datatype *q= new datatype[2*capacity];  
        int j = head; // copy filled array p (from head to tail) to array q[0...]  
        for (int i=0;i<capacity;i++) {q[i]=p[j]; j=j+1 % capacity;}  
        head = 0;  
        tail = capacity;    // pointing to the first empty slot for now  
        q[tail]=x;    // put x in the first empty slot  
        length++;    // account for adding x  
        capacity = 2*capacity; // reflect that capacity is now doubled  
        delete [] p;    // deletes the old array pointed to by p  
        p =q;    // makes p point to the new array.  
    }  
};
```

Time: $O(1)$ except at overflow, in which case $O(\text{length})$.

Implementation dequeue()

```
Queue::datatype Queue::dequeue( ){  
    assert(length>0);  
    datatype x= p[head];  
    head=head+1 % capacity;  
    return x;  
};
```

Time: $O(1)$.

END